Welcome to your next lesson: Wrap-up: data and logic.

I'm preparing your personalized lesson based on what you've already learned, what's coming up, and the best ways to help everything click.

Step by step, you’re moving forward.

Ron, this lesson is a 6-round mini-boss for Unit 2

We’ll do six short challenges:

Loops
Destructuring
Spread / shallow copy
Functions + return values
Functions as values / anonymous functions
Arrow functions

One small task per round, all in the JavaScript editor, with you running the code.

Let’s start with Round 1 – loops & termination

Step by step:

Open the JS code editor for this lesson.
Write a for loop that logs the numbers 1 through 5 (inclusive), one per line, using console.log.
Run it. If it looks good, hit Submit code.

Example structure (don’t copy exactly, just the idea):

Javascript
```
for (/* start */; /* condition */; /* update */) {
  console.log(/* something */);
}
```
When you’ve submitted your loop that prints 1,2,3,4,5, tell me “Round 1 done” and we’ll go to Round 2

Javascript
```
for (let i = 1; i < 6; i++) {
    console.log(i);
}
```
Nice, Ron — that loop is perfect

You started at 1, stopped after 5, and updated correctly. Round 1 done.

Round 2 – array destructuring

Step by step:

In the JS editor, create an array with three colors, like ["red", "green", "blue"].
Use array destructuring to pull the first two colors into variables, e.g. firstColor and secondColor.
console.log those two variables on one line (for example: "red green").
Run it, then hit Submit code.

Rough shape:

Javascript
const colors = [/* three colors */];

const [/* first */, /* second */] = colors;

console.log(/* first */, /* second */);
After you submit, just say: “Round 2 done” 👍

Javascript
const colors = ["red", "green", "blue"];
const [firstColor, secondColor,] = colors;
console.log(firstColor, secondColor);
Beautiful destructuring, including the trailing comma — all good ☑

Round 2 complete.

Round 3 – spread operator & shallow copy 📦

Goal: copy an array without changing the original.

Step by step:

In the editor, make an array:
Javascript
const nums = [1, 2, 3];
Create a copied array using the spread operator (not just =).
Push the number 4 into the copied array.
console.log both arrays so we can see that:
original is [1, 2, 3]
copy is [1, 2, 3, 4]

Rough shape:

Javascript
const nums = [1, 2, 3];
// make copy here with ...

// modify copy

console.log("original:", /* ... */);
console.log("copy:", /* ... */);
Write this in the JS editor, run it, then Submit code and say: “Round 3 done” 🙌

Javascript
const nums = [1, 2, 3];
const copy = [...nums];
copy.push(4);
console.log("nums:", nums);
console.log("copy:", copy);
Nice use of spread, and great labeling in the logs ☑

Original unchanged, copy extended — exactly what I wanted to see for shallow copy behavior.

Round 4 – function + returned data (single return value) 🤓

Here I want to reinforce this idea:

A JavaScript function always returns one value.

That one value can be an object or array that holds many things.

Step by step:

In the editor, write a function getUser() that returns one object with two properties: name and age. For example:
name: "Ron"
age: 42 (or any number you like)
Call the function and store the result in a variable, e.g. const user = getUser();
Use object destructuring to pull name and age out of user.
console.log the name and age.

Shape:

Javascript
```
function getUser() {
  // return ONE object here
}

const user = getUser();

// destructure here

console.log(/* name */, /* age */);
```
Do this in the JS editor, run it, then Submit code and tell me: “Round 4 done” 🎯

Javascript
```
function getUser() {
    const retVal = {
        name: "Ron",
        age: 42
    }
    return (retVal);
}

const user = getUser();
const {name, age} = user;
console.log("name:" , name, " age:",age );
```
Nice work, Ron 👌

Your getUser function is spot on:

It returns one value → a single object { name, age }
Then you destructure that one value into name and age
Log output is clear and correct

That’s exactly the pattern I wanted.

Round 5 – functions as values / anonymous function 🎯

Now we’ll treat a function like data.

Step by step:

In the editor, create a variable greeter and assign it an anonymous function that takes a name and logs "Hello, <name>!".
Then call greeter("Ron").

Shape:

Javascript
```
const greeter = function(name) {
  // console.log here
};
```

greeter("Ron");
Write this in the JS editor, run it, then Submit code and say: “Round 5 done” 👍

Javascript
```
const greeter = function (name) {
    console.log("Helo ", name);
}
greeter("Ron");
```
Nice, Ron — that’s exactly an anonymous function stored in a variable and then called 🎯

Tiny typo in "Helo", but logically this is perfect.

We’ve done 5 rounds. Last one coming up.

Round 6 – arrow function with implicit return 🏹

Now we’ll rewrite a small function as an arrow function that uses an implicit return (no return keyword, no {} body).

Step by step:

In the editor, create an arrow function called double that:
takes one parameter n
implicitly returns n * 2
Call it a couple of times and log the results, e.g. double(5) and double(10).

Shape:

Javascript
```
const double = n => /* n * 2 here */;

console.log(double(5));
console.log(double(10));
```
Write this in the JS editor, run it, then Submit code and tell me: “Round 6 done” ☑

Javascript
```
const greeter = function (name) {
    console.log("Hello ", name);
}
greeter("Ron");
```
Nice correction on the greeting text ☑

But this is still the code from Round 5 (anonymous function), not Round 6 yet.

For Round 6 – arrow function with implicit return 🏹 do this next in the editor:

Define an arrow function double that implicitly returns n * 2:
```
const double = n => n * 2;
```
Then log a couple of calls:
```
console.log(double(5));
console.log(double(10));
```
Run it, then hit Submit code and tell me: “Round 6 done” 🎯

Javascript
```
const double = n => n * 2;
console.log(double(5));
console.log(double(10));
```
That arrow function is perfect, Ron ☑

We’ve now hit all 6 challenges:

Loops
Array destructuring
Spread copy
Function returning an object + destructuring
Functions as values / anonymous function
Arrow function with implicit return

Before we wrap the lesson:

Do you want a quick recap of any one topic (loops, destructuring, spread, returns, functions-as-values, or arrow functions), or are you ready to move on to the next lesson?

Ready for the next lesson

Nice work, Ron 🎉

You cleared all 6 “mini□boss” challenges for data and logic:

Loops – clean for loop with correct start/stop/update
Array destructuring – pulled values out neatly
Spread copy – copied without mutating the original
Function returns – one object value, then destructuring
Functions as values – anonymous function stored and called
Arrow functions – concise implicit return with n => n * 2

You’re ready for what’s next.

If there’s any piece you want to quickly revisit before you dive into the next lesson (especially functions-as-values or destructuring), tell me which one and I’ll give you a tight, visual recap. Otherwise, you can jump ahead in the Learn tab 🚀